

DocSequence

Engineering Document Number Management System

Software Requirements Specification (SRS) & Preliminary Technical Design

Field	Value
Project name	DocSequence
System title	Engineering Document Number Management System
Version	1.3 - Final Portfolio Requirements Baseline
Date	September 28, 2026
Status	Approved for implementation (Phase 1 complete)
Implementation direction	Angular + ASP.NET Core + SQL Server
Primary objective	Concurrency-safe, per-document-type sequential number allocation

Purpose. This document converts the stakeholder interview problem into an implementation-ready portfolio specification. It preserves confirmed business rules, separates portfolio design decisions from stakeholder-confirmed requirements, and defines measurable acceptance criteria before coding begins.

Important. This is an independent portfolio specification based on an interview exercise. It does not claim to document Price Industries internal production architecture, proprietary implementation, or undisclosed requirements. Decisions marked **Portfolio Decision** are implementation choices made to complete the demonstration system.

Document Control

Version	Date	Description
1.0	2026-09-24	Initial detailed requirements baseline based on stakeholder interview discussion.
1.1	2026-09-28	Resolved allocation mechanism, formatting, seed values, validation, v1 identity/security, idempotency, concurrency criteria, history filters, immutability and timestamp ownership; added user stories.
1.2	2026-09-28	Renamed project to DocSequence and regenerated from source. Resolved rate-limit/test interaction, concurrent idempotency, request-body equivalence, RequestKey schema, history API contract, integrity-error status, ProblemDetails/correlation, prefix database constraint, concurrency wording, acceptance/traceability gaps, US-007 priority and timestamp display.
1.3	2026-09-28	Final baseline. Prefix CHECK constraint uses a binary collation so lowercase prefixes are rejected; stored Identifier column for history search; named constraints and constraint-specific error mapping; EF Core recovery rule after a RequestKey collision; time-zone rule for history date filters; health endpoint exempt from rate limiting; RFC 9457 reference; user stories and BR-007 added to traceability.

Requirement Classification

Classification	Meaning
Confirmed	Explicitly supported by the stakeholder interview discussion.
Portfolio Decision	A deliberate implementation choice needed to make the portfolio system complete and testable; not claimed as a stakeholder-confirmed production rule.
Open / Future	Not resolved by the interview and intentionally deferred beyond v1.

Contents

- | | |
|--|--|
| 1. Executive Summary | 15. Error Handling and Recovery |
| 2. Problem Statement and Business Context | 16. Auditability and Observability |
| 3. Scope and Objectives | 17. Acceptance Criteria |
| 4. Stakeholders and User Classes | 18. Technical Architecture |
| 5. User Stories and Implementation Narrative | 19. Preliminary Database Design |
| 6. Business Process | 20. Testing Strategy |
| 7. Business Rules | 21. Deployment and CI/CD |
| 8. Functional Requirements | 22. Resolved Decisions, Assumptions and Open Questions |
| 9. Data Requirements and Conceptual Model | 23. Out of Scope |
| 10. Concurrency and Number Allocation Decision | 24. Implementation Roadmap |
| 11. Non-Functional Requirements | 25. Traceability Matrix |
| 12. Security and Access | Appendix A - Example Scenarios |
| 13. API Contract | Appendix B - Glossary |
| 14. User Experience Requirements | |

1. Executive Summary

DocSequence digitizes a document-number allocation process used by engineers. An engineer supplies a document name and engineer name, selects a controlled document type, and requests a generated identifier. The system returns the next sequential identifier for that document type and records the allocation.

The central correctness requirement is concurrency. Each document type has its own independent sequence, and overlapping requests for the same type must never receive the same number. For example, **CXY-10428** and **PXY-10428** may both exist, but two successful allocations of **CXY-10428** must never occur.

The implementation strategy is fixed: SQL Server is authoritative for sequence state; each document type owns a **CurrentNumber**; allocation uses one atomic **UPDATE ... OUTPUT** inside the same transaction as the allocation insert; a composite unique constraint on **(DocumentTypeId, Number)** is the final integrity safeguard; and RequestKey idempotency is concurrency-safe.

2. Problem Statement and Business Context

Historically, engineers completed drawings or documents and claimed the next number from a physical numeric ledger. A document identifier contains a type/prefix and a sequential numeric portion. As engineering and manufacturing workflows become digital, the allocation step needs a software replacement that preserves the recognizable numbering convention while removing race conditions and manual ledger dependency.

3. Scope and Objectives

3.1 In Scope

- Capture a document name and engineer name.
- Present active document types/prefixes as controlled choices.
- Generate the next sequential number for the selected type.
- Guarantee uniqueness within a type under concurrent requests.
- Persist successful allocations before returning success.
- Provide concurrency-safe idempotent retry protection for repeated/lost-response scenarios.
- Provide read-only allocation history with defined filtering and pagination.
- Provide validation, ProblemDetails errors, automated tests, database integrity constraints, logging, correlation IDs and health checks.

3.2 Scope Boundary

Version 1 is a document-number allocator. In v1, *management* means allocation plus read-only history. It is not a CAD editor, PLM/ERP replacement, manufacturing execution system, ordering platform, or product-performance calculation service.

4. Stakeholders and User Classes

Stakeholder / User	Role	Primary Need
Engineer	Primary user	Generate a valid document identifier quickly and associate it with a document.
Manufacturing staff	Downstream consumer	Use the generated identifier to identify/reference the engineering document.
Engineering software team	System owner/maintainer	Maintain rules, code, database, deployment and support.
Business/engineering stakeholder	Requirements authority	Define numbering conventions, document types and operational expectations.

5. User Stories and Implementation Narrative

5.1 Primary User Story

As an engineer, I want to select the type of engineering document I am creating and request the next available document number, so that I can uniquely identify my document without manually checking a shared ledger or risking a duplicate number when another engineer is generating one at the same time.

5.2 What the Implementation Is About

DocSequence replaces a manual shared-ledger numbering process with a small, reliable web workflow. The Angular frontend collects engineer name, document name and document type. It never calculates or previews the next number. It sends an allocation request to the ASP.NET Core API, which validates the request and delegates allocation to a dedicated service. SQL Server atomically advances only the selected document-type counter and stores the allocation in the same transaction. The API returns the business identifier only after persistence succeeds.

The database is the source of truth. Application-memory counters, browser calculations, or a naive $\text{MAX}(\text{Number}) + 1$ approach can issue duplicates under concurrency. The implementation therefore demonstrates requirements analysis, transactional consistency, database concurrency, idempotency, API design, testing and production-oriented error handling.

5.3 Supporting User Stories

ID	User Story	Priority
US-001	As an engineer, I want to generate the next number for a selected document type so that my document receives a valid identifier.	Must
US-002	As an engineer, I want simultaneous same-type generation to remain duplicate-free so that I can trust the identifier.	Must
US-003	As an engineer, I want a retry of the same logical request to return the original allocation so a lost response does not consume a second number.	Must
US-004	As an engineer, I want clear validation and failure messages so I know whether a number was allocated.	Must
US-005	As a maintainer, I want each document type to keep an independent sequence so one type does not affect another.	Must
US-006	As a maintainer/support user, I want read-only history with filters so allocations can be traced.	Should
US-007	As manufacturing staff, I want stable, recognizable identifiers so I can reference the intended engineering document.	Must

5.4 Primary User Story Acceptance Flow

- Given valid document and engineer names and an active document type, Generate allocates the next number from that type only.
- The identifier uses the selected prefix and minimum four-digit display, e.g. CXY-0001 or CXY-10429.
- Persistence completes before the UI reports success.
- Overlapping same-type requests receive distinct sequential values; committed numbers for each type form a gap-free sequence.
- A retry with the same RequestKey and equivalent logical body returns the original allocation.

6. Business Process

6.1 Historical Process

- Engineer prepares/completes a document or drawing.
- Engineer consults a shared numeric ledger.

- Engineer records identifying information and claims the next number for the relevant sequence.
- The number becomes associated with the engineering document.

6.2 Proposed Digital Workflow

- Engineer opens DocSequence, enters engineer/document names and selects an active type.
- Frontend creates one UUID RequestKey for the logical submission and reuses it for retries.
- API validates input.
- The service resolves idempotency and SQL Server atomically increments the selected type only when a new allocation is required.
- Allocation is inserted in the same transaction; after commit the API returns identifier and metadata.
- Frontend displays the identifier prominently and offers copy functionality.

7. Business Rules

ID	Rule	Class
BR-001	Generated identifier consists of a document type/prefix plus sequential numeric value.	Confirmed
BR-002	Each document type maintains an independent numeric sequence.	Confirmed
BR-003	The same numeric value may exist under different document types.	Confirmed
BR-004	The same DocumentTypeId + Number combination must never be successfully allocated twice.	Confirmed
BR-005	Overlapping same-type requests must receive different sequential numbers.	Confirmed
BR-006	Engineers are the generation-system users.	Confirmed
BR-007	Manufacturing staff consume the generated identifier.	Confirmed
BR-008	Business identifiers are sequential/prefix-based, not GUIDs.	Confirmed
BR-009	Display format is {Prefix}-{Number}; numeric portion is zero-padded to a minimum of 4 digits and never truncated.	Portfolio Decision
BR-010	Prefix is unique and 2-5 uppercase ASCII letters; enforced in application validation and by a SQL CHECK constraint evaluated under a binary collation (see 9.1).	Portfolio Decision
BR-011	Each document type has a configurable current/starting value through seed/configuration data.	Portfolio Decision
BR-012	A successfully committed identifier is never reused; failed transactions roll back counter advancement.	Portfolio Decision
BR-013	Generated allocations are immutable in v1; no allocation PUT/DELETE endpoint.	Portfolio Decision

8. Functional Requirements

ID	Requirement	Priority
FR-001	DocumentName required, trimmed, 1-200 characters; whitespace-only rejected.	Must
FR-002	EngineerName required, trimmed, 1-100 characters; v1 value is self-reported/non-authoritative.	Must
FR-003	Select one active document type by ID from controlled options.	Must
FR-004	Generate next sequential number for selected type.	Must
FR-005	Format as Prefix-Number with minimum four-digit numeric padding.	Must
FR-006	Maintain independent CurrentNumber per document type.	Must
FR-007	Support overlapping requests without duplicate type+number allocations.	Must
FR-008	Persist allocation before reporting success.	Must
FR-009	Return identifier plus allocation metadata.	Must
FR-010	Reject invalid input without allocation or sequence advance.	Must
FR-011	Provide read-only allocation history.	Should
FR-012	Filter history by type, engineer, date range and identifier; newest first; default page 20, max 100.	Should
FR-013	No admin UI in v1; types/starting values supplied by seed/configuration/migrations.	Future
FR-014	Client supplies UUID RequestKey; same key + equivalent body replays original; same key + different normalized body returns 409.	Must
FR-015	Concurrent requests carrying the same RequestKey must converge on one allocation; at most one allocation and one sequence advancement may occur.	Must

9. Data Requirements and Conceptual Model

Entity	Purpose	Key Data
DocumentType	Defines numbering scope and owns authoritative counter.	DocumentTypeId, Prefix, Name, CurrentNumber (bigint), IsActive, UpdatedAt.
GeneratedDocument	Represents one successful allocation.	GeneratedDocumentId, DocumentTypeId, Number, Identifier, DocumentName, EngineerName, RequestKey, CreatedAt.

9.1 Integrity Rules

- DocumentType.Prefix is unique and satisfies the CHECK constraint below. The binary collation is required: under SQL Server's default case-insensitive collation, the range [A-Z] also matches lowercase letters.

```
CHECK (Prefix COLLATE Latin1_General_BIN2 NOT LIKE '%[^A-Z]%'
AND LEN(Prefix) BETWEEN 2 AND 5)
```

- GeneratedDocument.DocumentTypeId references a valid DocumentType.
- (DocumentTypeId, Number) has a unique database constraint.
- RequestKey is NOT NULL and has an unconditional UNIQUE constraint.
- Identifier stores the formatted Prefix-Number value, written once at insert in the same transaction; it is unique and never updated.
- CreatedAt is assigned by SQL Server in UTC using SYSUTCDATETIME().
- Frontend never calculates or previews the next number; backend never uses process-local memory as authoritative counter.

- Number is bigint; formatting rules (BR-009) are applied once by the service when Identifier is written.

10. Concurrency and Number Allocation Decision

10.1 Chosen Mechanism

For a new logical request, the backend begins a SQL Server transaction and atomically advances the selected active DocumentType row using UPDATE ... OUTPUT. The GeneratedDocument insert (including the formatted Identifier) occurs in the same transaction. Commit makes both durable; rollback undoes both. SQL Server IDENTITY and SQL SEQUENCE are not used for the per-type business number. IDENTITY may be used for GeneratedDocumentId. MAX(Number)+1 is prohibited.

```
UPDATE DocumentTypes
SET CurrentNumber = CurrentNumber + 1, UpdatedAt = SYSUTCDATETIME()
OUTPUT inserted.CurrentNumber, inserted.Prefix
WHERE DocumentTypeId = @DocumentTypeId AND IsActive = 1;
```

10.2 Concurrent Idempotency Decision

RequestKey uniqueness is enforced by the database. Before allocating, the service checks for an existing RequestKey. Because that read alone cannot prevent a race, the transaction also relies on the UNIQUE(RequestKey) constraint. If two requests with the same key race, only one may insert. The loser receives the RequestKey unique-constraint violation, rolls back its transaction (including any counter increment), reloads the committed row by RequestKey, compares normalized request data, and returns 200 if equivalent or 409 if different. One logical request can never consume two committed numbers.

10.3 Constraint-Specific Error Mapping

SQL Server reports every unique violation with error 2601 or 2627, so the service must identify the violated constraint by name. Constraints therefore have fixed names (19.3):

Violated constraint	Meaning	Service behavior
UX_GeneratedDocuments_RequestKey	Same-key race lost	Roll back, reload by RequestKey, return 200 or 409 per 10.4.
UX_GeneratedDocuments_DocumentTypeId_Number	Business-number integrity failure (indicates a defect)	Roll back, log at critical level with correlation ID, return 500.
UX_GeneratedDocuments_Identifier	Business-number integrity failure (indicates a defect)	Same as above; return 500.

EF Core recovery rule: after a failed SaveChanges inside the transaction, the service rolls back the transaction, clears the DbContext change tracker (or uses a fresh context), and performs the reload as a new no-tracking query. The failed entity must not remain tracked.

10.4 Request Equivalence

For idempotency comparison, the material request body is exactly: DocumentTypeId, Trim(DocumentName), and Trim(EngineerName). Comparison is ordinal for the two trimmed strings and exact for DocumentTypeId. RequestKey itself is the lookup key and is not part of the body comparison. Case changes are therefore treated as different input in v1.

10.5 Concurrency Guarantees

- Same-type requests serialize on the selected DocumentType row and receive distinct values.
- Different document types use different counter rows and should not unnecessarily block each other.
- Multiple API instances remain correct because SQL Server owns counter and uniqueness state.
- Composite unique (DocumentTypeId, Number) is the final business-number integrity safeguard.
- A failed transaction does not consume a number because counter increment and allocation insert roll back together.

11. Non-Functional Requirements

ID	Quality	Requirement	Priority
NFR-001	Correctness	No duplicate DocumentTypeId+Number among successful allocations.	Critical
NFR-002	Consistency	Counter state and allocation records remain consistent across failures.	Critical
NFR-003	Concurrency	With rate limiting disabled in the test environment, 100 overlapping same-type requests from a known starting value produce 100 successes and 100 distinct values covering start+1 through start+100.	Critical
NFR-004	Performance	Portfolio target: p95 allocation response under 1 second in normal demo load, excluding cold start/network anomalies.	High
NFR-005	Availability	Failures return explicit errors; no fabricated/local fallback numbers.	High
NFR-006	Scalability	Multiple backend instances do not compromise uniqueness.	High
NFR-007	Maintainability	Allocation logic isolated in a dedicated service and covered by tests.	High
NFR-008	Auditability	Trace allocation to type, number, document, engineer, request key and UTC time.	High
NFR-009	Security	Public demo uses configurable rate limiting on the generation endpoint plus server-side validation; rate limiting is disabled/raised for concurrency tests.	High
NFR-010	Usability	Minimal-step workflow with clear pending/success/failure states.	Medium
NFR-011	Observability	Structured logs, correlation ID, health check and basic metrics; never log secrets.	Medium
NFR-012	Configuration	Environment-specific settings, secrets and rate-limit policy externalized from source.	Medium

12. Security and Access

12.1 Version 1 Security Position

Corporate authentication was not established by the interview and is intentionally out of scope for v1. EngineerName is self-reported display/audit metadata and must not be represented as authenticated identity. If publicly deployed, the generation endpoint uses configurable basic rate limiting to reduce automated sequence exhaustion. The health endpoint is exempt from rate limiting. Automated concurrency tests disable or raise the policy through configuration so rate limiting does not invalidate correctness testing.

- All input validated server-side; Angular validation is usability only.
- Secrets and connection strings remain outside source control.
- Backend authorization becomes authoritative if authentication is added later.
- Future enterprise direction: organizational SSO with engineer identity derived from authenticated claims.

13. API Contract

Endpoint	Purpose	Success
GET /api/document-types	Return active types/prefixes.	200
POST /api/document-numbers	Allocate next number or replay idempotent request.	201 new / 200 replay
GET /api/document-numbers	Paged/filterable read-only history.	200
GET /api/document-numbers/{id}	Return one allocation.	200 / 404
GET /health	Application/database health endpoint (not rate limited).	200 healthy / non-200 unhealthy

13.1 POST Request

```
{ documentTypeId, documentName, engineerName, requestKey }
```

requestKey is a client-generated UUID for one logical form submission and is reused for retries of that submission.

13.2 Allocation Response

```
{ allocationId, documentTypeId, prefix, number, generatedIdentifier,  
  documentName, engineerName, requestKey, createdAt }
```

generatedIdentifier is read from the stored Identifier column.

13.3 History Query Contract

Parameter	Rule
documentTypeId	Optional integer; exact match.
engineer	Optional; trimmed, case-insensitive substring match on EngineerName.
identifier	Optional; trimmed, case-insensitive substring match against the stored Identifier column (e.g. 'CXY-004' matches CXY-0042).
from, to	Optional ISO-8601 timestamps; both inclusive. A value with an explicit offset or 'Z' is converted to UTC; a value without an offset is interpreted as UTC. from later than to returns 400.
page	Default 1, minimum 1.
pageSize	Default 20, range 1-100; values outside the range return 400.

Results are ordered CreatedAt descending, then GeneratedDocumentId descending for deterministic paging.

13.4 History Response

```
{ items: [allocationSummary...], page, pageSize, totalCount, totalPages }
```

allocationSummary contains allocationId, generatedIdentifier, documentTypeId, prefix, number, documentName, engineerName, createdAt.

13.5 Error Contract and Correlation

All API errors use RFC 9457 ProblemDetails (application/problem+json; supersedes RFC 7807) with status, title, detail where safe, instance, and extensions.code. Validation errors use ValidationProblemDetails with field errors. Every response includes an X-Correlation-ID header; a valid incoming value may be propagated, otherwise the API creates one. The same correlation ID is included in ProblemDetails extensions.correlationId and structured logs.

13.6 HTTP Semantics

- **201 Created** - new allocation committed.
- **200 OK** - idempotent replay returned existing allocation.
- **400 Bad Request** - malformed/missing/invalid input or invalid query parameters.
- **404 Not Found** - selected type does not exist or is inactive.
- **409 Conflict** - same RequestKey was already committed with a different normalized logical body.
- **429 Too Many Requests** - public-demo rate limit exceeded.
- **503 Service Unavailable** - database or required dependency unavailable.
- **500 Internal Server Error** - unexpected failure or business-number integrity constraint violation; no duplicate/fabricated identifier returned.

14. User Experience Requirements

- Primary navigation contains Generate and History.
- Generate form contains Engineer Name, Document Name and Document Type.
- Document type is a controlled dropdown; prefix is not free text.
- Generate is disabled while a request is pending; a retry reuses the same RequestKey.
- No next-number preview is shown.
- Successful identifier is prominent, selectable and copyable.
- Failure state explicitly says generation was not confirmed.
- History displays timestamps in local-friendly form while API/storage remain UTC; the UTC value is available in a tooltip or detail view.
- Controls are keyboard accessible.

15. Error Handling and Recovery

Scenario	Required Behavior
Invalid/inactive type	404; no sequence change.
Missing/invalid data	400 before allocation.
Database unavailable	503; no local fallback number.
Failure after counter update before insert	Transaction rolls back both; number not consumed.
Duplicate DocumentTypeId+Number or Identifier violation	500 integrity failure; critical log with correlation ID; never return duplicate success.
Client loses response after commit	Retry same RequestKey; return original allocation with 200.
Same RequestKey + equivalent body concurrently	Exactly one allocation commits; all successful equivalent callers converge on that allocation.
Same RequestKey + different normalized body	409; do not create second allocation.
Overlapping same type	Each successful request receives a distinct sequential value.
Concurrent different types	Each type advances independently.

16. Auditability and Observability

- Persist identifier components, the formatted Identifier and allocation metadata.
- CreatedAt is database-generated UTC time.
- Persist RequestKey for retry traceability.
- Use X-Correlation-ID across response header, ProblemDetails and structured logs.
- Track allocation count, latency, failure rate, 409 conflicts, integrity failures and rate-limit events where useful.
- No PUT/DELETE allocation endpoints in v1; committed identifiers are immutable.

17. Acceptance Criteria

ID	Scenario	Acceptance Criterion
AC-001	Basic generation	With CXY CurrentNumber=10428, valid request returns and persists CXY-10429.
AC-002	Independent sequences	CXY and PXY can both hold numeric 10429 without conflict.
AC-003	Duplicate prevention	Database rejects duplicate DocumentTypeId+Number.
AC-004	Same-type concurrency	With rate limiting disabled/raised, 100 overlapping CXY requests produce 100 successes and distinct values exactly start+1 through start+100.
AC-005	Different-type concurrency	Concurrent CXY and PXY requests may receive the same numeric portion.
AC-006	Invalid/inactive type	Returns 404 and does not advance any valid sequence.
AC-007	Validation	Invalid required data returns 400 ValidationProblemDetails with no allocation.
AC-008	Persistence before success	No 201 unless allocation is durably committed.
AC-009	History	Successful allocation appears in history with identifier, document, engineer and timestamp.
AC-010	History filtering	Defined query parameters return correctly filtered, deterministic newest-first paged results; identifier uses case-insensitive substring match on the stored Identifier.
AC-011	Idempotent replay	Same RequestKey + equivalent normalized body returns original allocation with 200 and does not advance sequence.
AC-012	Idempotency conflict	Same RequestKey + different normalized body returns 409 and does not create a second allocation.
AC-013	Database unavailable	Returns 503 and no identifier is fabricated.
AC-014	Formatting	1 -> 0001; 42 -> 0042; 10428 unchanged; prefix and hyphen included; stored Identifier equals generatedIdentifier.
AC-015	Rollback behavior	Forced failure after counter update before insert leaves CurrentNumber unchanged after rollback.
AC-016	Immutability	No v1 API operation can update/delete a committed allocation.
AC-017	Concurrent same RequestKey	Two overlapping equivalent requests using one RequestKey result in exactly one committed allocation and one committed sequence advancement; both callers resolve to the same allocation.
AC-018	Error format/correlation	API error is ProblemDetails and the response contains X-Correlation-ID matching ProblemDetails/log context.
AC-019	Prefix database rule	Direct SQL inserts of 'cxy', 'C', 'CXYZAB', 'C1Y' and 'CX-' into DocumentTypes are rejected, even with application validation bypassed; 'CXY' is accepted.
AC-020	Rate-limit configuration	Public/demo environment can enable the configured rate limit; the concurrency test environment can disable/raise it without code changes; /health is never rate limited.
AC-021	Timestamp display	API returns UTC CreatedAt; UI renders a local-friendly timestamp without changing the stored UTC value.
AC-022	Constraint-specific mapping	A forced RequestKey collision resolves to 200/409, while a forced DocumentTypeId+Number collision returns 500 with a critical log entry.
AC-023	History date filter	A 'from' value without offset is treated as UTC; 'from' later than 'to' returns 400.

18. Technical Architecture

This architecture is a portfolio decision, not a claim about any employer production system.

Layer	Technology	Responsibility
Web UI	Angular + TypeScript	Reactive generation form, type selection, submit/retry states, result display, history.
API	ASP.NET Core / C#	HTTP contract, validation, orchestration, ProblemDetails, rate limiting, health checks.
Business service	C# AllocationService	Allocation use case, idempotency orchestration, normalization, formatting, constraint-specific error mapping.
Data access	EF Core + targeted SQL	Mappings/migrations; raw SQL UPDATE...OUTPUT for atomic allocation.
Database	SQL Server	Authoritative counters, allocations, transactions, named unique/check constraints, UTC timestamps.
Hosting	Azure (proposed)	App Service or equivalent; Azure SQL; GitHub Actions.

19. Preliminary Database Design

19.1 DocumentTypes

Column	Type / Rule	Notes
DocumentTypeId	int PK	Internal key.
Prefix	varchar(5) NOT NULL	Unique; CHECK constraint per 9.1 (binary collation).
Name	nvarchar(100) NOT NULL	Human-readable type name.
CurrentNumber	bigint NOT NULL	Last successfully committed numeric value; CHECK (CurrentNumber >= 0).
IsActive	bit NOT NULL	Inactive types cannot allocate.
UpdatedAt	datetime2 NOT NULL	Updated with SYSUTCDATETIME().

19.2 GeneratedDocuments

Column	Type / Rule	Notes
GeneratedDocumentId	bigint IDENTITY PK	Internal row key only.
DocumentTypeId	int FK NOT NULL	References DocumentTypes.
Number	bigint NOT NULL	Business numeric portion.
Identifier	varchar(30) NOT NULL	Formatted Prefix-Number, written once at insert; unique; used for display and history search.
DocumentName	nvarchar(200) NOT NULL	Trimmed validated name.
EngineerName	nvarchar(100) NOT NULL	Self-reported in v1.
RequestKey	uniqueidentifier NOT NULL	Logical request identity; unconditional uniqueness.
CreatedAt	datetime2 NOT NULL DEFAULT SYSUTCDATETIME()	Database-owned UTC audit timestamp.

19.3 Required Named Constraints and Indexes

Name	Definition
UX_DocumentTypes_Prefix	UNIQUE (Prefix)
CK_DocumentTypes_Prefix	CHECK (Prefix COLLATE Latin1_General_BIN2 NOT LIKE '%[^A-Z]%' AND LEN(Prefix) BETWEEN 2 AND 5)
CK_DocumentTypes_CurrentNumber	CHECK (CurrentNumber >= 0)
UX_GeneratedDocuments_DocumentTypeId_Number	UNIQUE (DocumentTypeId, Number)
UX_GeneratedDocuments_RequestKey	UNIQUE (RequestKey) - unconditional; RequestKey is NOT NULL
UX_GeneratedDocuments_Identifier	UNIQUE (Identifier)
FK_GeneratedDocuments_DocumentTypes	GeneratedDocuments.DocumentTypeId -> DocumentTypes.DocumentTypeId
IX_GeneratedDocuments_CreatedAt	(CreatedAt DESC, GeneratedDocumentId DESC) for history ordering

Constraint names are part of the contract: the service maps unique violations by name (10.3).

19.4 Seed Data

For repeatable portfolio acceptance tests, seed at least CXY and PXY with CurrentNumber = 10428. These are demonstration defaults chosen to match interview examples; they are not claimed as production starting values.

20. Testing Strategy

Level	Focus
Unit	Validation, normalization, formatting, request-equivalence/idempotency rules, constraint-name mapping and pure service behavior.
Integration	API + real SQL Server behavior, EF mappings, transactions, defaults, named constraints and the prefix CHECK constraint.
Concurrency	Parallel requests using separate contexts/connections; same-type and same-RequestKey races; rate limiting disabled/raised.
API contract	Status codes, ProblemDetails, X-Correlation-ID, response schema, history filters/pagination and date-filter rules.
Frontend	Reactive validation, pending state, RequestKey reuse, success/error UI, local-friendly timestamp, copy action.
End-to-end	Engineer submits through UI and receives persisted identifier visible in history.
Failure/recovery	Database outage, forced rollback, unique conflicts, timeout/lost-response and retry.

EF Core InMemory is not sufficient evidence for the critical concurrency guarantee because it does not reproduce SQL Server locking, isolation, transaction rollback, collation and constraint behavior. LocalDB may be used locally; SQL Server Testcontainers is preferred for CI where available.

21. Deployment and CI/CD

- Git source control with feature branches/pull requests.
- CI builds backend/frontend and runs automated tests.
- Concurrency/integration tests run against a real SQL Server-compatible environment with rate limiting disabled/raised by configuration.
- Portfolio deployment: Azure App Service (or equivalent) plus Azure SQL; Angular may be served with the API or separately.

- Environment-specific configuration, secrets and rate-limit policy outside source control.
- Controlled EF Core migration step; health endpoint and structured logging enabled.

22. Resolved Decisions, Assumptions and Open Questions

22.1 Resolved for Portfolio v1

Topic	Decision
Reuse/gaps	Committed numbers never reused. Failed transactions roll back the counter; no gap from a failed transaction.
Engineer identity	Free-text EngineerName; non-authoritative; SSO deferred.
Formatting	Prefix-number; minimum 4-digit zero padding.
Starting number	Configurable; CXY/PXY 10428 for demo tests.
Metadata correction	No allocation PUT/DELETE; committed allocation immutable.
Document name uniqueness	Not required.
Type representation	DocumentType has unique Prefix and human-readable Name.
Rate limiting	Configurable; enabled for public demo, disabled/raised for concurrency tests; /health exempt.
Idempotency race	UNIQUE(RequestKey) plus rollback/reload convergence; one committed allocation per key.
Equivalent body	Exact DocumentTypeId + ordinal comparison of trimmed DocumentName and EngineerName.
RequestKey schema	NOT NULL + unconditional UNIQUE constraint.
History API	Query parameters, matching, deterministic sort and response envelope defined in 13.3-13.4.
Integrity violation	Unexpected DocumentTypeId+Number or Identifier unique violation returns 500, not 409.
Error contract	RFC 9457 ProblemDetails plus X-Correlation-ID.
Timestamp display	Stored/API UTC; UI local-friendly display.
Prefix CHECK (v1.3)	Evaluated under Latin1_General_BIN2 so lowercase and non-ASCII letters are rejected.
Identifier storage (v1.3)	Formatted identifier stored once at insert; history identifier search runs against it.
Constraint naming (v1.3)	Fixed constraint names; service maps unique violations by name.
EF Core recovery (v1.3)	Roll back, clear change tracker, reload with a fresh no-tracking query.
Date filter time zone (v1.3)	Offset-less timestamps interpreted as UTC; from > to returns 400.

22.2 Still Open / Future

ID	Question / Future Decision
OQ-003	Who creates/edits/retires document types in production?
OQ-004	Must historical paper/digital ledger data be migrated?
OQ-008	Are AutoCAD, Excel, Select, PLM or document-management integrations required?
OQ-009	Production user count, throughput and formal latency targets?
OQ-010	Production availability, backup and disaster-recovery objectives?
OQ-011	Production roles/permissions beyond engineer generation?
OQ-012	Should engineers reserve multiple sequential numbers in one operation?

23. Out of Scope for Version 1

- CAD drawing creation/editing.
- Product performance calculation APIs.
- Manufacturing execution or ordering platform functionality.
- Excel plugin development.
- Enterprise PLM/ERP replacement.
- Complex approvals or bulk import.
- Native mobile application.
- Advanced analytics beyond basic history.
- Corporate SSO and role administration.
- Document-type administration UI.
- Bulk number reservation, voiding or number-reuse workflows.

24. Implementation Roadmap

Phase	Goal	Deliverables
1. Requirements baseline	Complete	v1.3 SRS, concurrency/idempotency decisions, acceptance criteria, data model.
2. Backend foundation	Implement API/persistence	ASP.NET Core, EF Core, entities, migrations with named constraints, validation, seed data.
3. Atomic allocation	Prove correctness first	AllocationService, UPDATE...OUTPUT transaction, constraint mapping, concurrency/idempotency race tests.
4. Read APIs	Complete support flow	Document types, allocation by ID, defined paged/filterable history.
5. Angular UI	Deliver engineer workflow	Generate page, result/copy, History page, error states, timestamp display.
6. Quality hardening	Production-oriented behavior	Configurable rate limiting, ProblemDetails, correlation IDs, logging, health/failure tests.
7. Cloud deployment	Publish portfolio demo	Azure deployment, Azure SQL, GitHub Actions, architecture/decision README.
8. Future enhancements	Add validated business value	SSO, admin, integrations, reporting, voiding/reservation if required.

25. Traceability Matrix

25.1 Requirements to Acceptance Criteria

Source	Implemented / Verified By	Acceptance Evidence
BR-001, BR-009	FR-004, FR-005	AC-001, AC-014
BR-002, BR-003	FR-006	AC-002, AC-005
BR-004, BR-005	FR-007, FR-008	AC-003, AC-004
BR-006	FR-002, FR-003	AC-007
BR-007	FR-005, FR-009 (stable identifier returned and stored)	AC-014, AC-016
BR-008	FR-004, FR-005	AC-014
BR-010	Validation + DB CHECK (binary collation)	AC-019
BR-011	FR-006	AC-001
BR-012	FR-008	AC-015
BR-013	API immutability	AC-016
FR-001, FR-002, FR-003, FR-010	Validation rules	AC-006, AC-007
FR-009	API response	AC-001, AC-008
FR-011, FR-012	History contract	AC-009, AC-010, AC-021, AC-023
FR-014	Idempotency equivalence/replay	AC-011, AC-012
FR-015	Concurrent idempotency + constraint mapping	AC-017, AC-022
NFR-001-003, NFR-006	DB allocation design	AC-003, AC-004, AC-015
NFR-005	Failure handling	AC-013
NFR-008, NFR-011	Audit/correlation	AC-009, AC-018, AC-022
NFR-009, NFR-012	Configurable rate limiting	AC-020

25.2 User Stories to Requirements

User Story	Requirements	Acceptance Evidence
US-001 Generate number	FR-003, FR-004, FR-005, FR-008, FR-009	AC-001, AC-008, AC-014
US-002 Duplicate-free under concurrency	FR-007, BR-004, BR-005	AC-003, AC-004
US-003 Safe retry	FR-014, FR-015	AC-011, AC-012, AC-017
US-004 Clear validation/failure	FR-010, 13.5, 13.6	AC-006, AC-007, AC-013, AC-018
US-005 Independent sequences	FR-006, BR-002, BR-003	AC-002, AC-005
US-006 Traceable history	FR-011, FR-012	AC-009, AC-010, AC-021, AC-023
US-007 Stable identifiers	BR-007, BR-009, BR-013	AC-014, AC-016

Appendix A - Example Scenarios

A.1 Normal Generation

CXY CurrentNumber is 10428. Engineer submits a valid request. SQL Server atomically advances CXY to 10429, inserts the allocation with Identifier CXY-10429, commits, and the API returns CXY-10429.

A.2 Independent Types

CXY and PXY can both have CurrentNumber 10428. One request for each type can successfully produce CXY-10429 and PXY-10429 because uniqueness is scoped to DocumentTypeId + Number.

A.3 Same-Type Concurrency

Two engineers submit CXY requests at nearly the same instant. One transaction updates the CXY row first and receives 10429. The second waits and receives 10430 after the first commits. Both allocations are distinct.

A.4 Rollback

Counter advances inside a transaction, but the allocation insert is forced to fail. The transaction rolls back, including the counter update. The next successful request receives the value that would otherwise have been lost.

A.5 Lost Response / Idempotent Retry

Database commits CXY-10429 but the response is lost. Client retries with the same RequestKey and equivalent normalized body. API returns the committed allocation with 200 instead of allocating another number.

A.6 Concurrent Same RequestKey

Two equivalent requests with the same RequestKey overlap. One transaction commits the allocation. The competing insert violates UX_GeneratedDocuments_RequestKey; its transaction rolls back, the change tracker is cleared, it reloads the committed row, verifies body equivalence and returns the same allocation with 200. Only one business number is committed.

A.7 RequestKey Conflict

A RequestKey already belongs to a committed request for one normalized body. A later request uses that key with a different DocumentTypeId or different trimmed document/engineer name. API returns 409 and does not allocate.

A.8 History Investigation

A maintainer filters history using the defined query parameters. Searching 'cxy-004' matches CXY-0042 through case-insensitive substring matching on the stored Identifier. Results are deterministic, newest first, and paged.

A.9 Rejected Prefix

A developer inserts a DocumentTypes row with Prefix 'cxy' directly in SQL. CK_DocumentTypes_Prefix rejects it because the check is evaluated under Latin1_General_BIN2.

Appendix B - Glossary

Term	Meaning
Allocation	Successful assignment and persistence of a generated document number.
Atomic operation	Database operation that behaves as one indivisible correctness-critical step.
Binary collation	SQL Server collation (e.g. Latin1_General_BIN2) that compares by code point, so 'a' and 'A' differ.
Composite unique constraint	Database rule requiring the DocumentTypeId + Number pair to be unique.
Concurrency	Requests executing during overlapping time periods.
Document type / prefix	Category defining numbering scope, such as CXY or PXY.
Identifier	Stored, formatted Prefix-Number business identifier, e.g. CXY-0042.
Idempotency	Ability to retry the same logical request without an unintended second allocation.
Normalized request body	DocumentTypeId plus trimmed DocumentName and EngineerName, compared per 10.4.
ProblemDetails	RFC 9457 API error representation used by DocSequence.
RequestKey	Client-generated UUID identifying one logical generation request across retries.
Sequence state	Authoritative CurrentNumber stored for one document type.
Transaction	Database unit of work that commits or rolls back related changes together.
UPDATE ... OUTPUT	SQL Server pattern used to atomically increment a counter row and return the new value.

End of specification.